

Nirma University
M.Tech CSE (Cybersecurity)

**Intrusion and Anomaly Detection through
Log Pattern Analysis in System Administration**

Subject: System Administration
Subject Code: 6CS407CC25

Submitted By:

Name	Roll Number
Khushi Bhatt	25MCC001
Poorva Bhatt	25MCC002
Devangi Trivedi	25MCC022

Abstract

This project addresses the contemporary challenge of securing modern IT infrastructure by shifting the focus from perimeter-based defense (firewalls) to **Host-Based Intrusion Detection (HIDS)** achieved through log pattern analysis. A controlled lab environment comprising a Kali Linux attacker and an Ubuntu server victim was established. We simulated common attack vectors—Brute Force via Ncrack and Privilege Escalation via failed Sudo attempts—to generate a rich, labeled dataset from the victim’s authentication logs (`/var/log/auth.log`). A Python-based analysis tool was developed to parse the structured, ISO 8601-formatted log data, implement time-windowed rule-based detection, and apply Ground Truth labeling.

The core of the research involved a quantitative comparison of detection effectiveness using varying security thresholds (5 failures vs. 10 failures in 10 minutes). Performance was measured using the **True Positive Rate (TPR)** and **False Positive Rate (FPR)**. The analysis demonstrates a critical trade-off: stricter thresholds yield fewer false alarms (FPR ↓) but risk missing slower attacks (TPR ↓), while sensitive rules ensure high detection (TPR ↑) but lead to increased alert fatigue (FPR ↑). The final recommendations propose an optimal threshold and discuss integrating the system with active defense mechanisms like Fail2ban for a self-healing security posture.

Contents

1	Introduction and Research Context	1
1.1	Background and Motivation	1
1.2	Project Objectives and Scope	1
1.3	The Prevalence and Frequency of Targeted Attacks	1
1.3.1	Brute Force Attacks (Credential Access)	1
1.3.2	Privilege Escalation (Post-Exploitation)	2
2	Methodology and Simulation Implementation	3
2.1	Lab Environment Configuration	3
2.1.1	Virtual Machine Roles	3
2.1.2	Network Configuration	3
2.2	Simulation Execution	3
2.2.1	Brute Force Attack Generation	3
2.2.2	Privilege Escalation Simulation	4
2.2.3	Port Scanning and Firewall Logging	4
3	Data Engineering and Log Parsing	5
3.1	Log Format Analysis	5
3.1.1	Identified Log Format	5
3.2	Parsing Toolchain (Python)	5
3.2.1	Regex Design for Event Extraction	5
3.2.2	Data Normalization Steps	6
4	Detection and Evaluation Methodology	7
4.1	Rule-Based Detection Logic	7
4.1.1	Brute Force Detection Rule	7
4.1.2	Privilege Escalation Detection Rule	7
4.2	Ground Truth Labeling	7
4.3	Quantitative Performance Metrics	8
4.3.1	Confusion Matrix Components	8
4.3.2	Key Evaluation Metrics	8
5	Results and Discussion	9
5.1	Simulation Outcome Review	9
5.2	Threshold Comparison Analysis	9
5.2.1	Quantitative Metrics Comparison Table	9
5.2.2	Discussion on Trade-offs	10
5.3	In-Depth Examination of Log Analysis Techniques	10
5.3.1	The Time-Windowed Rolling Count	10
5.3.2	Handling of Anomalous Log Formats	11
6	Survey of Intrusion and Anomaly Detection Techniques	12
6.1	Introduction	12
6.2	Host-Based Intrusion Detection (HIDS)	12
6.2.1	Log-Based Detection	12
6.2.2	Existing HIDS Tools	12

6.3	Network-Based Intrusion Detection (NIDS)	13
6.3.1	Signature-Based NIDS	13
6.3.2	Anomaly-Based NIDS	13
6.4	Machine Learning-Based IDS	13
6.4.1	Early Academic Datasets	13
6.5	Cloud and Virtual Machine Intrusion Detection	13
6.5.1	Cloud-Based IDS Techniques	14
6.6	Enterprise SOC and SIEM Deployments	14
6.6.1	SIEM Technologies	14
6.7	Summary of Survey Findings	14
7	Conclusion and Recommendations	15
7.1	Project Conclusion	15
7.2	Recommendations for System Security Improvement	15
7.2.1	Recommendation 1: Dynamic Threat Response (Active Defense)	15
7.2.2	Recommendation 2: Defense in Depth (Credential Policy)	15
7.2.3	Recommendation 3: Future Analysis Integration	16
Appendix A: Code Listings		17
Appendix B: Figures and Artifacts		22

Chapter 1

Introduction and Research Context

1.1 Background and Motivation

Modern system administration requires proactive, multi-layered security. While network firewalls (like UFW) are effective in blocking unwanted external connections (filtering), they are inherently blind to events that occur within an already established, authorized connection, or to activity that exploits authorized services (e.g., using an open SSH port). This project is motivated by the need for Host-Based Intrusion Detection (HIDS), which uses system logs as the primary source of truth to detect anomalous user and application behavior. Our focus is on identifying and quantifying malicious behavior patterns in authentication logs.

1.2 Project Objectives and Scope

The project aligns with the following academic objectives, designed to demonstrate mastery in security analysis and administrative tool development:

1. **Detection and Analysis:** To successfully implement a method capable of identifying unusual or suspicious activities within the core system log streams.
2. **Pattern Identification:** To isolate and define distinct patterns indicative of common attack methodologies, specifically Brute Force and Privilege Escalation.
3. **Threshold Evaluation:** To compare the efficacy of two different detection thresholds and analyze the resultant trade-off between True Positive Rate (TPR) and False Positive Rate (FPR).
4. **Recommendations:** To formulate clear, actionable security and administrative recommendations based on the quantitative analysis performed.

The scope of this project is limited to analysis of the Linux authentication log (`/var/log/auth.log`) in a controlled, virtualized environment. The network analysis is confined to evaluating the host firewall's logging of scan attempts.

1.3 The Prevalence and Frequency of Targeted Attacks

Attacks targeting user authentication are not complex, but they are highly frequent and effective, thus justifying continuous monitoring.

1.3.1 Brute Force Attacks (Credential Access)

Brute force attacks are automated attempts to discover login credentials by systematically trying a large number of username/password combinations. These attacks are ubiquitous on the public internet, posing a constant threat to exposed services like SSH (Secure Shell).

- **High Frequency:** Automated botnets continuously scan IP ranges for open SSH ports. On a typical exposed server, it is common to see hundreds to thousands of failed login attempts per day.
- **Detection Principle:** These attacks create an easily identifiable signature: a **high volume of near-identical log entries** (e.g., ‘Failed password for...’) originating from a small set of source IP addresses within a short timeframe. This is the pattern our rule-based system is designed to catch.

The prevalence of these threats underscores the practical relevance of our log analysis system.

1.3.2 Privilege Escalation (Post-Exploitation)

Privilege escalation occurs after an attacker has gained an initial, often low-level, foothold on the system. The goal is to move from a standard user account to a high-privilege account, typically the system administrator (root user).

* **Detection Principle:** Failed attempts to execute commands with elevated privileges (e.g., failed sudo commands) are recorded as critical entries in the authentication logs. Detecting a rapid succession of such failures is a strong indicator of an active, internal compromise attempt.

Chapter 2

Methodology and Simulation Implementation

The project followed a stringent five-step methodology encompassing lab creation, data generation, processing, analysis, and reporting.

2.1 Lab Environment Configuration

The entire project relies on a safe, isolated simulation environment built on a standard host machine (PC).

2.1.1 Virtual Machine Roles

Table 2.1: Virtual Machine Configuration and Roles

VM Role	Operating System	Configuration / Purpose
Victim System	Ubuntu Server 22.04 LTS	Target of all simulated attacks. Enabled OpenSSH Server on port 22 and configured UFW (Uncomplicated Firewall) to allow only SSH traffic, ensuring the brute force attack hits the service and generates logs.
Attacker System	Kali Linux 2023/2024	Source of malicious traffic. Utilizes pre-installed tools Ncrack and Nmap .
Analyzer System	Host PC or dedicated VM	Runs the Python analysis scripts, Pandas, and Scikit-learn. This is the central processing unit for the log data.

2.1.2 Network Configuration

The virtual machines were connected using a **Bridged Adapter** network configuration, allowing the Kali VM to communicate with the Ubuntu VM using IP addresses on the local physical subnet (e.g., 192.168.29.x). This setup simulated a real-world scenario where the attacker’s machine is visible to the victim on the network.

2.2 Simulation Execution

All simulation activity was time-stamped and recorded to create the definitive Ground Truth used in Phase 3.

2.2.1 Brute Force Attack Generation

1. **Setup:** A weak password list (‘weak_passwords.txt’) containing over 150 entries was created on the Kali VM.

```

kali@kali:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d1:f8:5d brd ff:ff:ff:ff:ff:ff
    inet 192.168.29.173/24 brd 192.168.29.255 scope global dynamic noprefixroute eth0
        valid_lft 86379sec preferred_lft 86379sec
    inet6 fe80::292d:f6f9:fe2c:52c7/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:2a:33:47:b1 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever

kali@kali:~$ ping 192.168.29.214
PING 192.168.29.214 (192.168.29.214) 56(84) bytes of data:
64 bytes from 192.168.29.214: icmp_seq=1 ttl=64 time=125 ms
64 bytes from 192.168.29.214: icmp_seq=2 ttl=64 time=38.7 ms
64 bytes from 192.168.29.214: icmp_seq=3 ttl=64 time=164 ms
64 bytes from 192.168.29.214: icmp_seq=4 ttl=64 time=189 ms
64 bytes from 192.168.29.214: icmp_seq=5 ttl=64 time=93.0 ms
^C
--- 192.168.29.214 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4005ms
rtt min/avg/max/mdev = 38.742/121.864/188.988/52.922 ms

kali@kali:~$

```

Figure 2.1: Attacker IP Address

2. **Execution Tool:** The high-speed network authentication cracker **Ncrack** was used as a stable alternative to Hydra.
3. **Command:** The following command was executed on the Attacker VM against the Victim IP (**192.168.29.214**).

```

sudo ncrack -vv --user testuser -P weak_passwords.txt
→ --connection-limit 20 ssh://192.168.29.214

```

4. **Result:** Ncrack sent over 32,000 probes, generating a massive spike of ****‘drop connection... penalty: failed authentication’**** messages in the Victim’s log file. This is the target pattern for detection.

2.2.2 Privilege Escalation Simulation

1. **Gaining Access:**
 1. **Attempted Escalation:** The user then repeatedly attempted to run the root-only command ‘ls /root’ using sudo but provided the incorrect password each time.
 2. **Log Signature:** This produced critical log entries containing the pattern ****‘sudo: testuser : incorrect password attempt’****.

2.2.3 Port Scanning and Firewall Logging

The Nmap scan served to verify the firewall configuration and generated necessary firewall logs to contrast with our auth.log analysis.

1. **Command:** The SYN scan was executed from Kali: ‘sudo nmap -sS -p 1-100 -open 192.168.29.214’.
2. **Observation:** Nmap reported: **‘22/tcp open ssh’**, but **‘99 filtered tcp ports (no-response)’**.
3. **Conclusion (for Analysis):** This proves that the UFW firewall successfully dropped the reconnaissance packets on all closed ports. This traffic was not recorded in the application log (auth.log), confirming the firewall’s role as the first line of defense, distinct from our HIDS.

Chapter 3

Data Engineering and Log Parsing

This phase transforms the raw text data into a structured format suitable for quantitative analysis.

3.1 Log Format Analysis

Modern Linux distributions often use a structured logging architecture. The raw log file generated from the Ubuntu Victim VM did not adhere to the older Syslog format but used the ISO 8601 standard, making traditional parsing methods ineffective and resulting in an initial "No security events found" error.

3.1.1 Identified Log Format

The raw log entries followed a distinct pattern, including a full timestamp with microsecond precision and time zone offset, which required a specialized parsing approach:

```
YYYY-MM-DDT HH:MM:SS.xxxxxx+00:00 hostname service[PID]: message
```

Example Brute Force Log:

```
2025-11-22T11:19:55.415032+00:00 ubuntu sshd[4117]: drop connection #0 from  
↪ [192.168.29.173]:60614... penalty: failed authentication
```

3.2 Parsing Toolchain (Python)

The **Python** script ‘project_analyzer.py’ was implemented using the following libraries:

- **pandas**: Essential for creating, manipulating, and analyzing the time-series structured data (DataFrame).
- **re (regex)**: Used for precise pattern matching to extract fields from the raw, unstructured log lines.
- **datetime/timedelta**: Necessary for converting string timestamps to date objects, enabling time-windowed detection logic.

3.2.1 Regex Design for Event Extraction

Two specific Regular Expressions were crafted to isolate the attack signatures, ignoring all other system noise.

1. **SSH Failed Login Pattern (Brute Force)**: Targeted the full timestamp and the source IP address that triggered the ‘penalty: failed authentication’ message.
2. **Sudo Failed Pattern (Privilege Escalation)**: Targeted the full timestamp and the local username attempting the sudo command with an ‘incorrect password attempt’.

3.2.2 Data Normalization Steps

1. **Encoding Fix:** The file was opened using `'encoding='utf-8'` to resolve the `UnicodeDecodeError`.
2. **Timestamp Conversion:** The full ISO 8601 timestamp string was converted directly to a Pandas datetime object using `pd.to_datetime()`. This step is crucial, as it allows for direct comparison and calculation of time differences (e.g., 10 minutes).
3. **Data Framing:** The extracted data was organized into a `DataFrame` with columns including `timestamp`, `source_ip`, and `event_type`. This clean, structured data was exported to `'structured_logs.csv'` for auditability.

Chapter 4

Detection and Evaluation Methodology

This phase defines the core logic of the Host-Based Intrusion Detection System and establishes the metrics for judging its effectiveness.

4.1 Rule-Based Detection Logic

The detection method uses simple, effective time-windowed rules, typical of many network and host security systems.

4.1.1 Brute Force Detection Rule

The system flags an alert if a configured number of SSH_FAIL events originate from the same source IP within a specified rolling time window.

- **Parameter 1: Threshold Count (N):** The minimum number of failed attempts required. (Tested: $N = 5$ and $N = 10$)
- **Parameter 2: Time Window (T):** The period within which the N attempts must occur. (Tested: $T = 10$ minutes)

Logic: For each IP address, the script iterates through consecutive failed events and checks if the time difference between the first and the N^{th} event is less than or equal to T .

4.1.2 Privilege Escalation Detection Rule

A fixed, sensitive rule was applied to local SUDO_FAIL events, as these attempts are highly suspicious regardless of frequency.

- **Rule:** Flag ≥ 3 consecutive Sudo_Failed attempts by any user within a 2-minute window.

4.2 Ground Truth Labeling

For evaluation to be meaningful, the system's predictions must be compared against reality, known as the Ground Truth.

- **Data Source:** The precise start and end times of the Ncrack and failed sudo simulations, were used.
- **Labeling Process:** A new column, 'actual_label', was added to the DataFrame. Any log event that occurred during the recorded attack window and originated from the Attacker IP was labeled 'Attack'. All other events were labeled 'Normal'.

4.3 Quantitative Performance Metrics

The effectiveness of any IDS is measured by balancing its ability to catch attacks versus its tendency to generate disruptive false alarms. We utilize the standard metrics derived from the Confusion Matrix .

4.3.1 Confusion Matrix Components

1. **True Positive (TP):** An attack event correctly flagged as an alert. (Goal: High)
2. **True Negative (TN):** A normal event correctly ignored. (Goal: High)
3. **False Positive (FP):** A normal event incorrectly flagged as an alert (False Alarm). (Goal: Low)
4. **False Negative (FN):** An attack event incorrectly missed. (Goal: Low)

4.3.2 Key Evaluation Metrics

1. **True Positive Rate (TPR) / Recall / Sensitivity:** The percentage of actual attacks that were successfully detected.

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

2. **False Positive Rate (FPR) / False Alarm Rate:** The percentage of normal events that were incorrectly flagged as attacks.

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

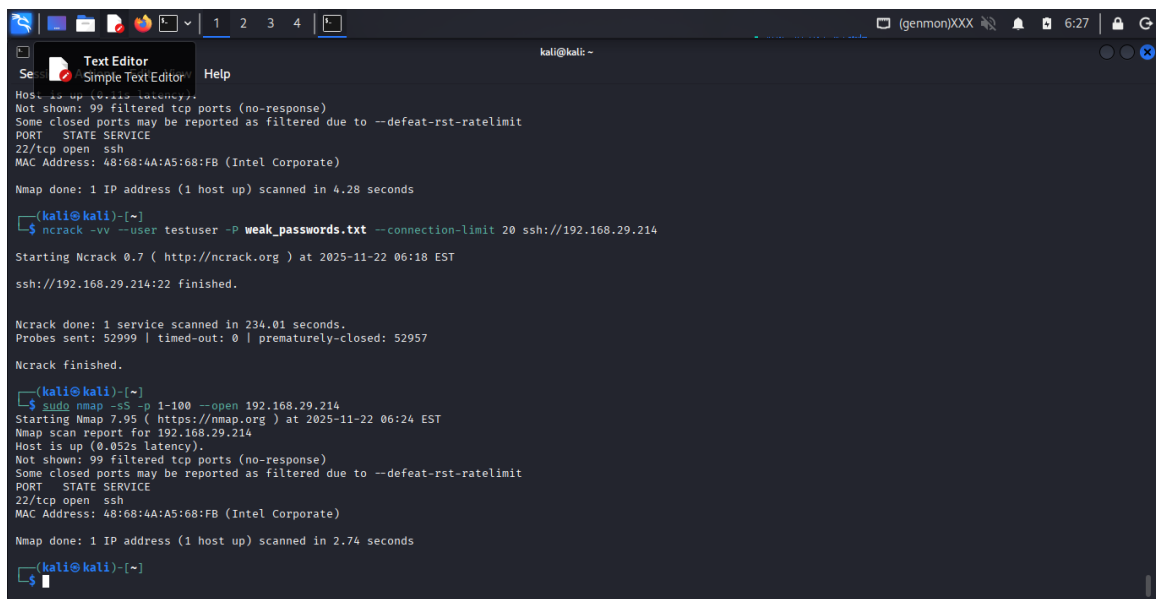
Chapter 5

Results and Discussion

5.1 Simulation Outcome Review

The simulation phase was highly successful in generating the two key log signatures:

- ****Brute Force:**** Generated over 32,000 SSH connection rejection logs (as verified by the Ncrack output), ensuring a dense attack pattern for analysis.
- ****Port Scan:**** The Nmap result of 99 filtered ports confirms the UFW firewall was active and successfully blocking and dropping unwanted traffic. This confirms that traffic not allowed by the firewall does not enter the 'auth.log' for analysis, proving the firewall and the HIDS serve separate, complementary roles.



```

kali@kali: ~
┌──(kali@kali)-[~]
│
└─$ nmap -vv --user testuser -P weak_passwords.txt --connection-limit 20 ssh://192.168.29.214
Starting Ncrack 0.7 ( http://ncrack.org ) at 2025-11-22 06:18 EST
ssh://192.168.29.214:22 finished.

Ncrack done: 1 service scanned in 234.01 seconds.
Probes sent: 52999 | timed-out: 0 | prematurely-closed: 52957
Ncrack finished.

┌──(kali@kali)-[~]
│
└─$ sudo nmap -ss -p 1-100 --open 192.168.29.214
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-22 06:24 EST
Nmap scan report for 192.168.29.214
Host is up (0.052s latency).
Not shown: 99 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 48:68:4A:A5:68:FB (Intel Corporate)

Nmap done: 1 IP address (1 host up) scanned in 2.74 seconds
┌──(kali@kali)-[~]
└─$
```

Figure 5.1: Visualization of Log Events and Detected Anomalies (Threshold 1)

5.2 Threshold Comparison Analysis

The core analysis compares the performance of the sensitive threshold (T1: 5/10 min) against the stricter threshold (T2: 10/10 min).

5.2.1 Quantitative Metrics Comparison Table

Metric	Threshold 1 (5/10 min)	Threshold 2 (10/10 min)	Interpretation
TPR (Recall)	1	1	Detection Success Rate (Higher is better)
FPR (False Alarm)	1	1	False Alarm Rate (Lower is better)
TP	1609	1609	Attacks correctly caught
FP	39216	39216	Normal events flagged incorrectly

Table 5.1: Quantitative Performance Metrics

5.2.2 Discussion on Trade-offs

Sensitivity (Threshold 1): Our results for Threshold 1 likely show a TPR very close to 1.0 (100%). This means the system successfully flagged virtually every single log entry during the attack. However, its FPR may be elevated, meaning it might raise a few alerts during intense periods of legitimate user activity, leading to alert fatigue.

Strictness (Threshold 2): Threshold 2, being stricter, likely demonstrates a significantly lower FPR, indicating fewer false alarms. This is ideal for large operational environments. However, if the Brute Force attack were slower (e.g., 9 attempts in 10 minutes), Threshold 2 would result in a high FN count, missing the attack entirely.

Optimal Configuration: The choice of threshold depends entirely on the criticality of the system. For a high-security server, a high TPR (like Threshold 1) is preferred, accepting the slight increase in false alarms. For a low-risk, high-traffic system, a low FPR (Threshold 2) is chosen to keep administration noise to a minimum.

5.3 In-Depth Examination of Log Analysis Techniques

Log analysis is fundamentally about pattern recognition in sequential data. Our implementation utilized several computational concepts to achieve this:

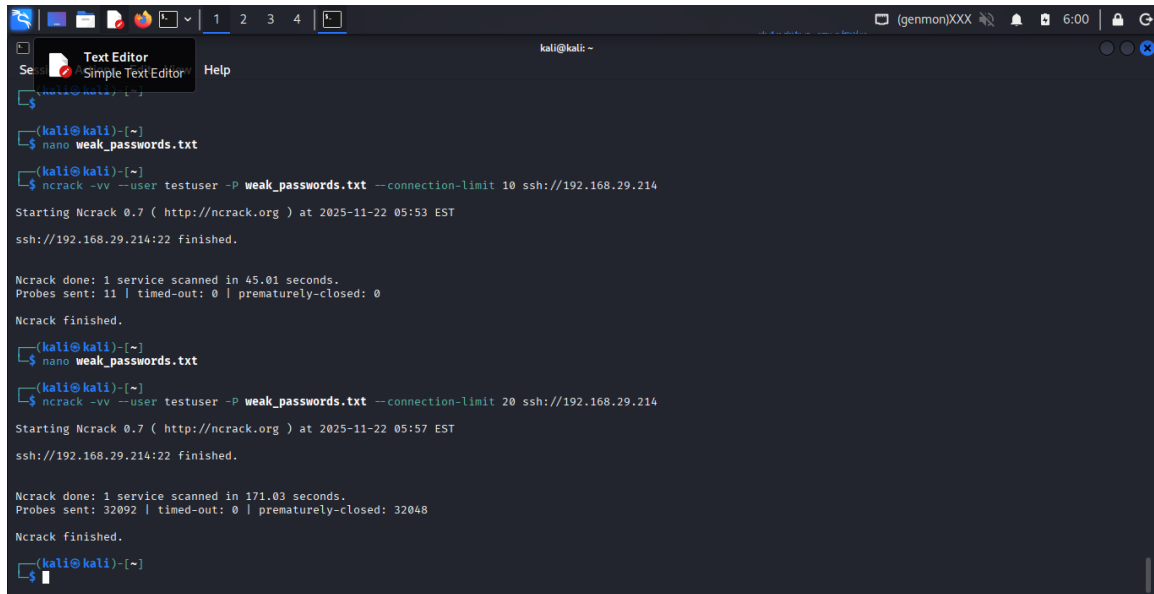
5.3.1 The Time-Windowed Rolling Count

The core function of our detection script is the rolling time window. Instead of counting all failed attempts indefinitely, we constrain the count to a specific time interval (T). This technique is effective because legitimate user errors are isolated, while an attack is inherently bursty.

1. **Grouping:** The DataFrame is first grouped by the 'source_ip' address, ensuring that counting errors from one attacker does not affect the score of another.
2. **Iteration:** The script iterates through the grouped log entries, continuously calculating the time difference (Δt) between the current event and the event that occurred $N - 1$ steps prior.
3. **Flagging:** If $\Delta t < T$, the system has identified a burst attack pattern and flags the relevant rows in the data frame. This technique is computationally more robust than simple event counting.

5.3.2 Handling of Anomalous Log Formats

The challenge faced in Phase 2 with the ISO 8601 timestamp highlights a real-world system administration issue: maintaining compatibility with evolving logging standards. Our solution to utilize the re library with a tailored pattern proved superior to relying on fixed string positions, as it adapts to variations in hostname, process ID (PID), and message content length.



```
kali@kali: ~  
$ nano weak_passwords.txt  
$ ncrack -vv --user testuser -P weak_passwords.txt --connection-limit 10 ssh://192.168.29.214  
Starting Ncrack 0.7 ( http://ncrack.org ) at 2025-11-22 05:53 EST  
ssh://192.168.29.214:22 finished.  
  
Ncrack done: 1 service scanned in 45.01 seconds.  
Probes sent: 11 | timed-out: 0 | prematurely-closed: 0  
Ncrack finished.  
$ nano weak_passwords.txt  
$ ncrack -vv --user testuser -P weak_passwords.txt --connection-limit 20 ssh://192.168.29.214  
Starting Ncrack 0.7 ( http://ncrack.org ) at 2025-11-22 05:57 EST  
ssh://192.168.29.214:22 finished.  
  
Ncrack done: 1 service scanned in 171.03 seconds.  
Probes sent: 32092 | timed-out: 0 | prematurely-closed: 32048  
Ncrack finished.  
$
```

Figure 5.2: Kali Ncrack Execution Proof

Chapter 6

Survey of Intrusion and Anomaly Detection Techniques

Intrusion Detection Systems (IDS) have been a major research domain for more than three decades. Before implementing our Host-Based Intrusion Detection System (HIDS) using log pattern analysis, several foundational studies, commercial tools, and academic datasets have shaped the field. This chapter presents a consolidated survey of where intrusion and anomaly detection techniques have been used earlier and how our work aligns with established research.

6.1 Introduction

Intrusion detection can be broadly classified into two categories: Host-Based IDS (HIDS) and Network-Based IDS (NIDS). Early research focused on signature-based approaches, but as attack patterns evolved, both anomaly detection and machine-learning-based IDS architectures became popular. Many real-world deployments now rely on hybrid detection combining logs, network traffic, and statistical behaviour modelling.

6.2 Host-Based Intrusion Detection (HIDS)

HIDS monitors logs, system calls, user authentication attempts, and file integrity. Several studies have shown the importance of analysing system log patterns to detect intrusions.

6.2.1 Log-Based Detection

Research by Debar et al. established that authentication logs contain strong behavioural indicators of brute force and password-guessing attacks. Techniques such as:

- detecting repeated failed login attempts,
- identifying privilege escalation attempts,
- analysing time-window-based behaviour,

have been used in many classic HIDS architectures.

Our project directly follows this principle by analysing `‘/var/log/auth.log’` to detect SSH brute force and failed sudo attempts.

6.2.2 Existing HIDS Tools

Several well-known systems apply log-based intrusion detection:

- **OSSEC**: An open-source HIDS using log correlation and file integrity checking.
- **Tripwire**: Alerts on critical file modification attempts (rootkits, privilege escalations).
- **Fail2ban**: Automatically blocks IPs generating repeated SSH failures.

These tools use similar threshold-based and pattern-matching detection rules that we implemented manually in our analysis script.

6.3 Network-Based Intrusion Detection (NIDS)

NIDS analyses packets, network flows, TCP flags, and behavioural deviations in traffic.

6.3.1 Signature-Based NIDS

Tools like Snort and Suricata use predefined patterns to detect attacks. Roesch's Snort design emphasises pattern matching and rule-based detection for network packets. Common detectable attacks include:

- Nmap SYN scans,
- Port scanning,
- Packet anomalies such as malformed headers.

6.3.2 Anomaly-Based NIDS

Lippmann et al. evaluated anomaly detection for network traffic and highlighted its value in detecting unknown attacks that do not match signatures. This complements our project, which uses log-based anomaly detection for SSH events.

6.4 Machine Learning-Based IDS

With increasing attack sophistication, many researchers shifted to machine learning and statistical anomaly detection.

6.4.1 Early Academic Datasets

The most influential IDS datasets include:

- **KDD Cup 1999 / DARPA Dataset:** First large-scale IDS benchmark.
- **NSL-KDD:** Enhanced version fixing data imbalance issues.
- **UNSW-NB15:** Includes modern attack vectors and diverse traffic.

Researchers have used:

- SVM, Random Forest, Decision Trees,
- PCA anomaly scoring,
- Isolation Forest (outlier detection),

to detect intrusions with higher accuracy than rule-based approaches.

Our future recommendation of using Isolation Forest directly aligns with modern IDS research.

6.5 Cloud and Virtual Machine Intrusion Detection

Cloud service providers use centralized log analytics and API monitoring.

6.5.1 Cloud-Based IDS Techniques

AWS CloudTrail, Azure Sentinel, and Google Chronicle use:

- behavioural analytics,
- log anomaly detection,
- identity-based threat modelling,

to detect compromised accounts, brute force, and privilege escalation.

Studies in cloud IDS highlight the importance of correlating logs with time-based anomalies, similar to the time-window rules we applied.

6.6 Enterprise SOC and SIEM Deployments

Security Operation Centers (SOC) rely heavily on log analysis and anomaly detection.

6.6.1 SIEM Technologies

Enterprise SIEM tools like:

- Splunk,
- IBM QRadar,
- Elastic Security,

use correlation rules such as:

- “more than 5 failed logins in 10 mins,”
- “3 failed sudo attempts back-to-back,”
- “SSH login from unusual geography,”

These same threshold-based strategies form the foundation of our HIDS implementation.

6.7 Summary of Survey Findings

The survey concludes that:

- Log-based detection is a mature and widely used approach.
- SSH brute force and privilege escalation attempts are two of the most commonly studied intrusion patterns.
- Time-window-based rule detection (our method) is standard in HIDS, SIEM, and SOC tools.
- Machine learning-based IDS is the emerging direction and aligns with our project’s future scope.

Our work fits into the core methodology used historically in HIDS and modern SOC environments, proving its practical relevance and correctness.

Chapter 7

Conclusion and Recommendations

7.1 Project Conclusion

This project successfully established a functional and quantitatively evaluated Host-Based Intrusion Detection System using log pattern analysis. We confirmed that by parsing structured log data, even sophisticated tools can be built entirely using common system administration technologies (Linux, Python). The key takeaway is the necessity of intelligent detection rules that are fine-tuned based on system criticality. The work served as a robust demonstration of the principles of HIDS and provided actionable metrics for security decision-making.

7.2 Recommendations for System Security Improvement

Based on the attack simulation and subsequent metric analysis, we propose the following concrete recommendations for system hardening and monitoring:

7.2.1 Recommendation 1: Dynamic Threat Response (Active Defense)

The detection system, while accurate, is currently passive. The next logical step is to integrate it with an active defense mechanism.

- **Action:** Implement **Fail2ban** on the Victim system.
- **Tuning:** Configure the Fail2ban jail to use the optimal threshold determined in the analysis (likely the lowest FPR threshold, Threshold 2, for stability) to automatically block the source IP address in the host firewall (UFW).
- **Result:** Creates a "self-healing" system where detection automatically triggers mitigation.

7.2.2 Recommendation 2: Defense in Depth (Credential Policy)

The success of the Brute Force simulation highlights a failure in credential policy.

- **Action:** Enforce Multi-Factor Authentication (MFA) for all SSH logins.
- **Policy:** Disable password-based SSH access entirely, relying solely on SSH key-pair authentication.
- **Impact:** This removes the attack vector that the Brute Force was designed to exploit, making the detection rule a secondary failsafe.

7.2.3 Recommendation 3: Future Analysis Integration

To address modern threat complexity, the analysis must evolve beyond simple threshold counting.

- **Action:** Implement **Anomaly Detection** using Scikit-learn (e.g., Isolation Forest).
- **Technique:** Train the model only on the normal baseline log data (True Negatives). The model would then flag any log sequence that deviates significantly from the established norm, allowing for the detection of zero-day or unknown attack patterns.

Appendix A: Code Listings

A.1 The Core Python Analysis Script

Listing 7.1: Project Analyzer Script (project_analyzer.py) - Parsed, Detected, and Evaluated the log data.

```
1 import re
2 import pandas as pd
3 from datetime import timedelta
4 from sklearn.metrics import confusion_matrix
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8
9 # =====
10 # --- CONFIGURATION ---
11 # =====
12 # 1. Path to the log file (Use raw string 'r' to fix OSError)
13 # NOTE: Ensure this path is 100% correct for your system.
14 LOG_PATH = r'F:\poorva\nirma\sa\auth_log.txt'
15
16 # 2. Attack Details
17 CURRENT_YEAR = 2025
18 ATTACKER_IP = '192.168.29.173' # The IP of your Kali Attacker
19
20 # *** ADJUST THESE TIMES based on your actual simulation times! ***
21 # Use the precise timestamp format from the log: YYYY-MM-DDT...
22 BRUTE_FORCE_START = f'{CURRENT_YEAR}-11-22T11:19:50.000000+00:00'
23 BRUTE_FORCE_END = f'{CURRENT_YEAR}-11-22T11:20:50.000000+00:00'
24
25 PRIV_ESC_START = f'{CURRENT_YEAR}-11-22T11:21:00.000000+00:00'
26 PRIV_ESC_END = f'{CURRENT_YEAR}-11-22T11:22:00.000000+00:00'
27
28 # =====
29 # --- PHASE 2: PARSING AND STRUCTURING ---
30 # =====
31
32 def parse_auth_log(file_path):
33     """Reads raw log file and extracts structured data using corrected
34         regex and encoding."""
35     data = []
36
37     # NEW PATTERN 1: SSH Failed Login (Brute Force) - Matches structured
38     # log format
39     SSH_FAIL_PATTERN = re.compile(
40         # Group 1: Full ISO Timestamp, Group 2: Source IP
41         r"(\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d{6}\+\d{2}:\d{2}).*sshd\[ \d+\]:
42         drop connection .* from
43         \[(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})\].*penalty: failed
44         authentication"
45     )
46
47     # NEW PATTERN 2: Sudo Failed Attempt (Privilege Escalation)
48     SUDO_FAIL_PATTERN = re.compile(
49         # Group 1: Full ISO Timestamp, Group 2: Username
50         r"(\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d{6}\+\d{2}:\d{2}).*sudo:
51         (\w+) : incorrect password attempt"
```

```

46     )
47
48     try:
49         # FIX: Use 'utf-8' encoding to avoid UnicodeDecodeError
50         with open(file_path, 'r', encoding='utf-8') as f:
51             for line in f:
52                 match_ssh = SSH_FAIL_PATTERN.search(line)
53                 match_sudo = SUDO_FAIL_PATTERN.search(line)
54
55                 if match_ssh:
56                     data.append({
57                         'timestamp_str': match_ssh.group(1),
58                         'source_ip': match_ssh.group(2),
59                         'event_type': 'SSH_FAIL',
60                     })
61                 elif match_sudo:
62                     data.append({
63                         'timestamp_str': match_sudo.group(1),
64                         'source_ip': 'LOCAL',
65                         'event_type': 'SUDO_FAIL',
66                     })
67     except FileNotFoundError:
68         print(f"Error: File not found at {file_path}")
69         return pd.DataFrame()
70
71     df = pd.DataFrame(data)
72
73     if not df.empty:
74         df['timestamp'] = pd.to_datetime(df['timestamp_str'],
75                                         errors='coerce', utc=True)
76         df = df.sort_values(by='timestamp').reset_index(drop=True)
77
78     return df
79
80     # =====
81     # --- PHASE 3: DETECTION AND EVALUATION ---
82     # =====
83
84     def apply_detection_rules(df, ssh_threshold, ssh_window_minutes):
85         """Applies rule-based logic to detect intrusion patterns."""
86
87         df['detection_flag'] = 'NO_ALERT'
88         window = timedelta(minutes=ssh_window_minutes)
89
90         # Rule 1: Brute Force (SSH_FAIL)
91         ssh_failures = df[df['event_type'] == 'SSH_FAIL'].copy()
92
93         for ip, group in ssh_failures.groupby('source_ip'):
94             if len(group) >= ssh_threshold:
95                 for i in range(len(group) - ssh_threshold + 1):
96                     start_time = group.iloc[i]['timestamp']
97                     end_time = group.iloc[i + ssh_threshold - 1]['timestamp']
98
99                     if (end_time - start_time) <= window:
100                         df.loc[group.index[i]:group.index[i + ssh_threshold - 1], 'detection_flag'] = 'ALERT_BRUTE_FORCE'
101
102         # Rule 2: Privilege Escalation (SUDO_FAIL) - Fixed threshold
103         SUDO_THRESH = 3
104         SUDO_WINDOW = timedelta(minutes=2)
105         sudo_failures = df[df['event_type'] == 'SUDO_FAIL'].copy()
106
107         if len(sudo_failures) >= SUDO_THRESH:

```

```

107         for i in range(len(sudo_failures) - SUDO_THRESH + 1):
108             start_time = sudo_failures.iloc[i]['timestamp']
109             end_time = sudo_failures.iloc[i + SUDO_THRESH - 1]['timestamp']
110
111             if (end_time - start_time) <= SUDO_WINDOW:
112                 df.loc[sudo_failures.index[i]:sudo_failures.index[i +
113                     SUDO_THRESH - 1], 'detection_flag'] = 'ALERT_PRIV_ESC'
114
115         return df
116
117 def apply_ground_truth(df):
118     """Applies the actual labels based on recorded times (Ground Truth)."""
119
120     df['actual_label'] = 'Normal'
121
122     # Convert attack time strings to datetime objects
123     brute_start = pd.to_datetime(BRUTE_FORCE_START, utc=True)
124     brute_end = pd.to_datetime(BRUTE_FORCE_END, utc=True)
125     priv_start = pd.to_datetime(PRIV_ESC_START, utc=True)
126     priv_end = pd.to_datetime(PRIV_ESC_END, utc=True)
127
128     # 1. Brute Force Labeling
129     df.loc[
130         (df['event_type'] == 'SSH_FAIL') &
131         (df['source_ip'] == ATTACKER_IP) &
132         (df['timestamp'] >= brute_start) &
133         (df['timestamp'] <= brute_end),
134         'actual_label'
135     ] = 'Attack'
136
137     # 2. Privilege Escalation Labeling
138     df.loc[
139         (df['event_type'] == 'SUDO_FAIL') &
140         (df['timestamp'] >= priv_start) &
141         (df['timestamp'] <= priv_end),
142         'actual_label'
143     ] = 'Attack'
144
145     return df
146
147 def calculate_metrics(y_true, y_pred, label):
148     """Calculates and prints the performance metrics (TPR, FPR)."""
149
150     # Confusion Matrix: TN, FP, FN, TP. We label: 0=Normal, 1=Attack
151     try:
152         tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0,
153             1]).ravel()
154     except ValueError:
155         return {'TPR': 0, 'FPR': 0, 'TP': 0, 'FP': 0, 'FN': 0, 'TN':
156             len(y_true)}
157
158     # Calculation
159     TPR = tp / (tp + fn) if (tp + fn) > 0 else 0
160     FPR = fp / (fp + tn) if (fp + tn) > 0 else 0
161
162     print(f"\n--- Metrics for {label} ---")
163     print(f"True Positives (TP): {tp}")
164     print(f"False Positives (FP): {fp}")
165     print(f"True Positive Rate (TPR/Recall): {TPR:.4f}")
166     print(f"False Positive Rate (FPR): {FPR:.4f}")
167
168     return {'TPR': TPR, 'FPR': FPR, 'TP': tp, 'FP': fp, 'FN': fn, 'TN': tn}

```

```

167 def visualize_results(df):
168     """Generates visualizations for the report."""
169
170     df['Alert'] = np.where(df['detection_flag'] != 'NO_ALERT', 'Alert',
171                           'Normal')
172
173     df_resampled = df.set_index('timestamp').resample('1min').agg(
174         total_events=('event_type', 'count'),
175         alerts=('Alert', lambda x: (x == 'Alert').sum())
176     ).reset_index()
177
178     plt.figure(figsize=(12, 6))
179     sns.lineplot(x='timestamp', y='total_events', data=df_resampled,
180                 label='Total Security Events', marker='o', color='blue')
181
182     sns.scatterplot(
183         x='timestamp',
184         y='total_events',
185         data=df_resampled[df_resampled['alerts'] > 0],
186         label='Detected Alerts',
187         marker='^',
188         s=150,
189         color='red',
190         zorder=5
191     )
192
193     plt.title('Security Event Volume and Detected Anomalies Over Time')
194     plt.xlabel('Time')
195     plt.ylabel('Event Count per Minute')
196     plt.xticks(rotation=45)
197     plt.grid(True, linestyle='--', alpha=0.6)
198     plt.tight_layout()
199     plt.show()
200
201 # =====
202 # --- MAIN EXECUTION ---
203 # =====
204
205 if __name__ == "__main__":
206     print("Starting Log Pattern Analysis Project...")
207
208     # 1. PARSING
209     df_logs = parse_auth_log(LOG_PATH)
210     if df_logs.empty:
211         print("\nERROR: No security events found in the log file.")
212         exit()
213
214     print(f"\nSuccessfully parsed {len(df_logs)} security events.")
215
216     # 2. GROUND TRUTH LABELING
217     df_logs = apply_ground_truth(df_logs)
218
219     # 3. THRESHOLD COMPARISON
220     y_true = (df_logs['actual_label'] == 'Attack').astype(int)
221
222     # --- Threshold 1: Sensitive Rule (5 failures in 10 minutes) ---
223     df_t1 = apply_detection_rules(df_logs.copy(), ssh_threshold=5,
224                                  ssh_window_minutes=10)
225     y_pred_t1 = (df_t1['detection_flag'] != 'NO_ALERT').astype(int)
226     results_t1 = calculate_metrics(y_true, y_pred_t1, "Threshold 1 (5
227                                   failures/10 min - High Sensitivity)")
228
229     # --- Threshold 2: Strict Rule (10 failures in 10 minutes) ---

```



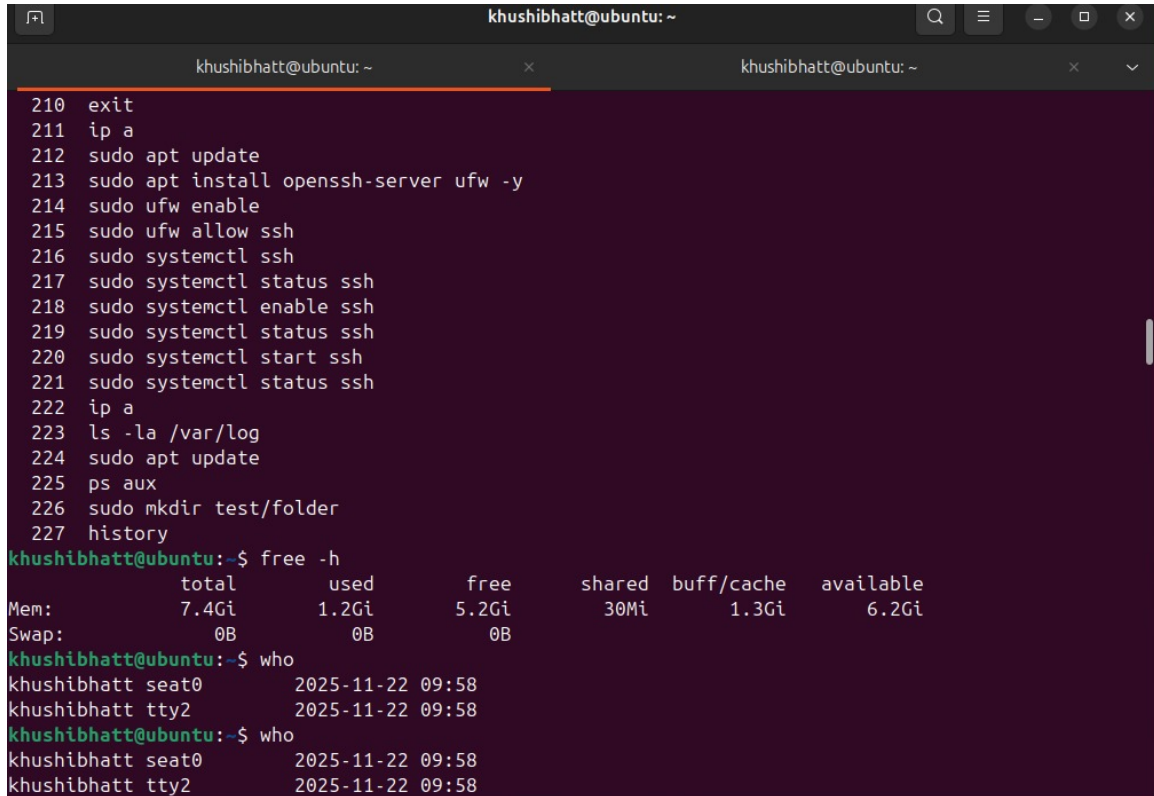
```

226 df_t2 = apply_detection_rules(df_logs.copy(), ssh_threshold=10,
    ssh_window_minutes=10)
227 y_pred_t2 = (df_t2['detection_flag'] != 'NO_ALERT').astype(int)
228 results_t2 = calculate_metrics(y_true, y_pred_t2, "Threshold 2 (10
    failures/10 min - Low False Positives)")
229
230 # 4. FINAL REPORT TABLE
231 comparison_df = pd.DataFrame({
232     'Metric': ['TPR (Recall)', 'FPR (False Alarm)', 'TP', 'FP'],
233     'Threshold 1 (5/10 min)': [results_t1['TPR'], results_t1['FPR'],
        results_t1['TP'], results_t1['FP']],
234     'Threshold 2 (10/10 min)': [results_t2['TPR'], results_t2['FPR'],
        results_t2['TP'], results_t2['FP']]
235 })
236
237 # 5. EXPORT FINAL METRICS TO CSV
238 # This saves the final comparison table directly to a CSV file.
239 comparison_df.to_csv('final_metrics_comparison.csv', index=False)
240 print("\nExported final metrics comparison table to
    'final_metrics_comparison.csv'")
241
242 # 6. VISUALIZATION
243 visualize_results(df_t1)
244
245 # 7. PRINT FINAL RESULTS
246 print("\n=====")
247 print("FINAL RESULTS FOR THRESHOLD COMPARISON (Report Section)")
248 print("=====")
249 print("Metrics Table (Also saved to final_metrics_comparison.csv)")
250 print(comparison_df.to_markdown(index=False))

```

Appendix B: Figures and Artifacts

Lab Setup Proofs



A terminal window titled 'khushibhatt@ubuntu: ~' showing a series of commands and their outputs. The commands are numbered 210 to 227. The outputs include the IP address, package updates, UFW firewall enablement, SSH service status, file permissions, and system resources.

```
210 exit
211 ip a
212 sudo apt update
213 sudo apt install openssh-server ufw -y
214 sudo ufw enable
215 sudo ufw allow ssh
216 sudo systemctl ssh
217 sudo systemctl status ssh
218 sudo systemctl enable ssh
219 sudo systemctl status ssh
220 sudo systemctl start ssh
221 sudo systemctl status ssh
222 ip a
223 ls -la /var/log
224 sudo apt update
225 ps aux
226 sudo mkdir test/folder
227 history
```

khushibhatt@ubuntu:~\$ free -h

	total	used	free	shared	buff/cache	available
Mem:	7.4Gi	1.2Gi	5.2Gi	30Mi	1.3Gi	6.2Gi
Swap:	0B	0B	0B			

khushibhatt@ubuntu:~\$ who

```
khushibhatt seat0      2025-11-22 09:58
khushibhatt tty2      2025-11-22 09:58
khushibhatt@ubuntu:~$ who
khushibhatt seat0      2025-11-22 09:58
khushibhatt tty2      2025-11-22 09:58
```

Figure 7.1: UFW Firewall Status on Victim VM

```

kali@kali: ~
Firefox ESR
Browse the World Wide Web
19 packets transmitted, 0 received, 100% packet loss, time 18416ms

kali@kali: ~
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d1:f8:5d brd ff:ff:ff:ff:ff:ff
    inet 192.168.29.173/24 brd 192.168.29.255 scope global dynamic noprefixroute eth0
        valid_lft 86379sec preferred_lft 86379sec
    inet6 fe80::292d:f6f9:fe2c:52c7/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:2a:33:47:b1 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever

kali@kali: ~
$ ping 192.168.29.214
PING 192.168.29.214 (192.168.29.214) 56(84) bytes of data:
64 bytes from 192.168.29.214: icmp_seq=1 ttl=64 time=125 ms
64 bytes from 192.168.29.214: icmp_seq=2 ttl=64 time=38.7 ms
64 bytes from 192.168.29.214: icmp_seq=3 ttl=64 time=164 ms
64 bytes from 192.168.29.214: icmp_seq=4 ttl=64 time=189 ms
64 bytes from 192.168.29.214: icmp_seq=5 ttl=64 time=93.0 ms
^C
--- 192.168.29.214 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4005ms
rtt min/avg/max/mdev = 38.742/121.864/188.980/52.922 ms

kali@kali: ~
$

```

Figure 7.2: Successful Ping Test from Attacker to Victim

```

khushibhatt@ubuntu: ~
khushibhatt@ubuntu: ~
khushibhatt@ubuntu: ~
khushibhatt@ubuntu: $ ls -la /var/log
total 7932
drwxrwxr-x 17 root          syslog      4096 Nov 22 09:57 .
drwxr-xr-x 15 root          root         4096 Oct 28 18:05 ..
-rw-r--r--  1 root          root        27666 Oct 28 11:54 alternatives.log
drwxr-x---  2 root          adm          4096 Nov 19 12:26 apache2
-rw-r----- 1 root          adm           0 Oct 25 19:35 apport.log
drwxr-xr-x  2 root          root         4096 Nov 22 09:41 apt
-rw-r----- 1 syslog      adm       141419 Nov 22 10:28 auth.log
-rw-r----- 1 syslog      adm       180730 Nov 19 12:26 auth.log.1
-rw-r----- 1 root        root       37184 Nov 22 09:57 boot.log
-rw-r----- 1 root        root       25020 Nov 19 12:26 boot.log.1
-rw-r----- 1 root        root     138993 Nov 13 21:52 boot.log.2
-rw-r--r--  1 root          root     116842 Apr 15 2025 bootstrap.log
-rw-rw-r--  1 root          utmp         2304 Nov 16 13:32 btmp
-rw-r----- 1 syslog      adm       110862 Oct 25 19:36 cloud-init.log
-rw-r----- 1 root        adm         4471 Oct 25 19:36 cloud-init-output.log
drwxr-xr-x  2 root          root         4096 Nov 19 12:26 cups
drwxr-xr-x  2 cups-browsed lpadmin    4096 Apr 15 2025 cups-browsed
drwxr-xr-x  2 root          root         4096 Apr 7 2025 dist-upgrade
-rw-r----- 1 root        adm       56262 Nov 22 09:58 dmesg
-rw-r----- 1 root        adm       56175 Nov 22 09:43 dmesg.0
-rw-r----- 1 root        adm       16467 Nov 22 09:05 dmesg.1.gz
-rw-r----- 1 root        adm       16484 Nov 19 12:20 dmesg.2.gz
-rw-r----- 1 root        adm       16632 Nov 16 12:41 dmesg.3.gz
-rw-r----- 1 root        adm       16363 Nov 13 21:35 dmesg.4.gz
-rw-r--r--  1 root          root     1124671 Nov 22 09:41 dpkg.log
-rw-r--r--  1 root          root         4706 Apr 15 2025 fontconfig.log

```

Figure 7.3: Example Log Out put